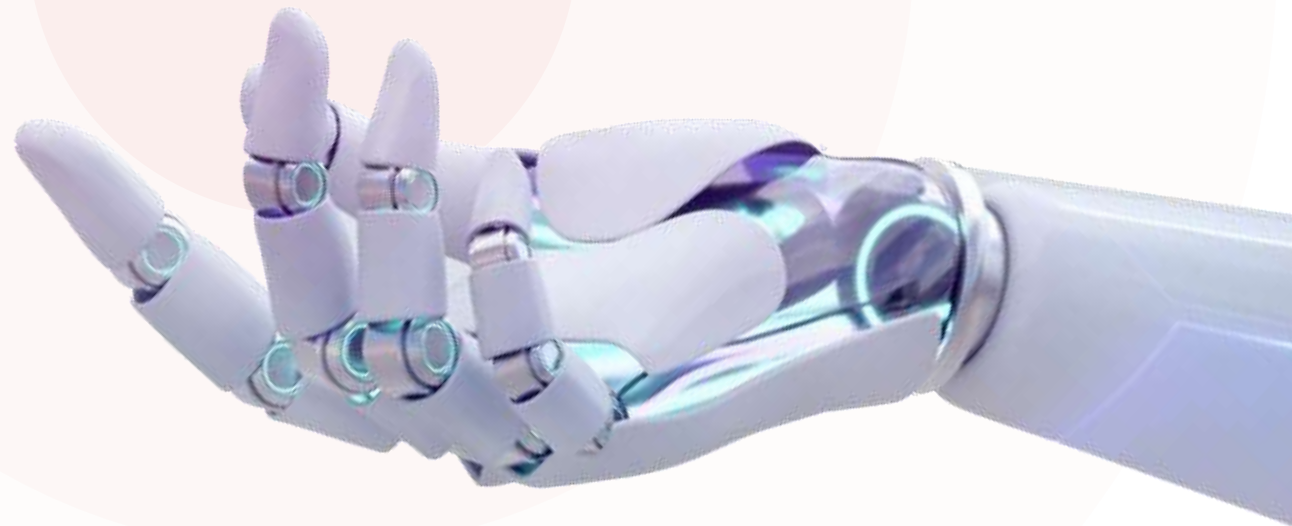


程序流程控制

控制程序的执行顺序（重点内容）



多一句没有，少一句不行，用更短的时间，教会更实用的技术



程序的三种执行顺序

顺序结构

自上而下的执行程序

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println("A");  
        System.out.println("B");  
        System.out.println("C");  
    }  
}
```

分支结构

根据条件，选择对应的代码执行

if、switch

循环结构

控制某段代码重复的执行

for、while、do-while



程序的执行顺序

BMI (Body Mass Index, 身体质量指数) 是用来评估体重是否适宜的一个常用指标。它通过体重 (以千克为单位) 除以身高 (以米为单位) 的平方来计算。

BMI的正常范围通常定义为18.5到24.9 (kg/m^2)。具体来说:

- BMI小于18.5被认为是体重过低;
- BMI在18.5到24.9之间被认为是正常范围;
- BMI在25到29.9之间被认为是超重;
- BMI在30及以上被认为是肥胖。

// 根据天气温度区间输出穿什么衣服



```
public static void getClothes(int temp) {  
    if (temp < 0) {  
        System.out.pr  
    } else if (temp >= 0) {  
        System.out.pr  
    } else if (temp >= 10) {  
        System.out.pr  
    } else if (temp >= 20) {  
        System.out.pr  
    } else {  
        System.out.pr  
    }  
}
```



CONTENT

程序流程控制

- 分支结构
 - if
 - switch
- 循环结构
- break、continue
- 综合小案例



if 分支结构

- 根据条件的真或假，来决定执行某段代码。

if 分支应用场景



BMI (Body Mass Index, 身体质量指数) 是用来评估体重是否适宜的一个常用指标。它通过体重 (以千克为单位) 除以身高 (以米为单位) 的平方来计算。

BMI的正常范围通常定义为18.5到24.9 (kg/m^2)。具体来说:

- BMI小于18.5被认为是体重过低;
- BMI在18.5到24.9之间被认为是正常范围;
- BMI在25到29.9之间被认为是超重;
- BMI在30及以上被认为是肥胖。





if 分支结构

if 分支有三种形式

```
if (条件表达式) {  
    代码;  
}
```

```
if (条件表达式) {  
    代码1;  
} else {  
    代码2;  
}
```

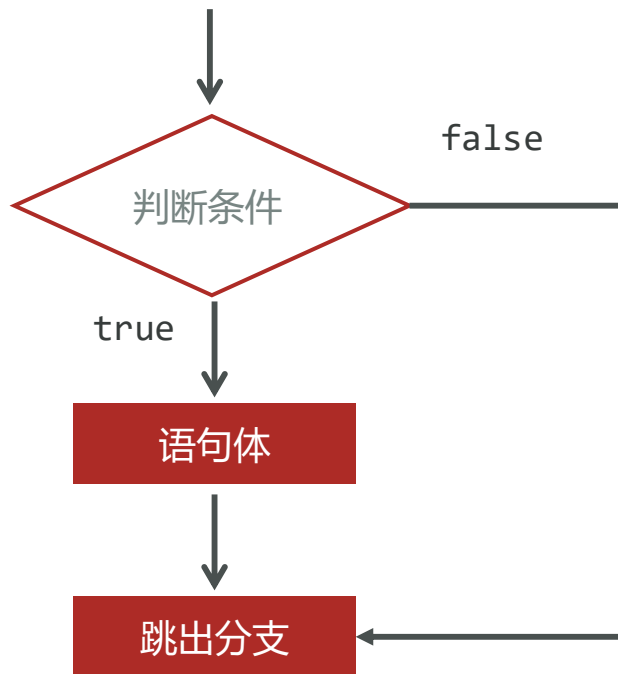
```
if (条件表达式1) {  
    代码1;  
} else if (条件表达式2) {  
    代码2;  
} else if (条件表达式3) {  
    代码3;  
}  
...  
else {  
    代码n;  
}
```

if 分支结构

if 第一种形式

格式:

```
if (条件) {  
    语句体;  
}  
...
```



执行流程:

① 首先判断条件表达式的结果, 如果为 **true** 执行语句体, 为 **false** 就不执行语句体。

注意事项:

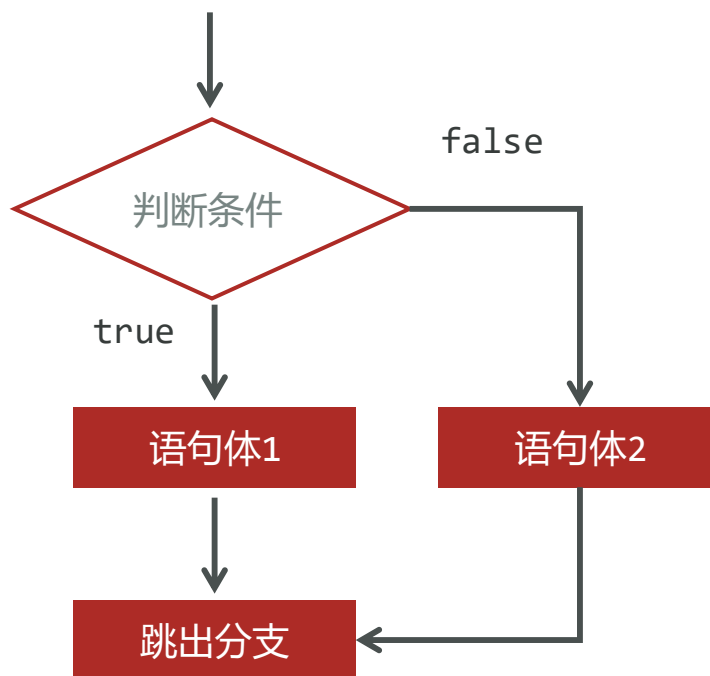
- `if(条件){}`, `()` 后不能跟 `;` 否则 `{}` 中的代码将不受 `if` 的控制了。
- `if` 语句中, 如果大括号控制的只有一行代码, 则大括号可以省略不写。

if 分支结构

if 第二种形式

格式:

```
if (条件) {  
    语句体1;  
} else {  
    语句体2;  
}  
...
```



执行流程:

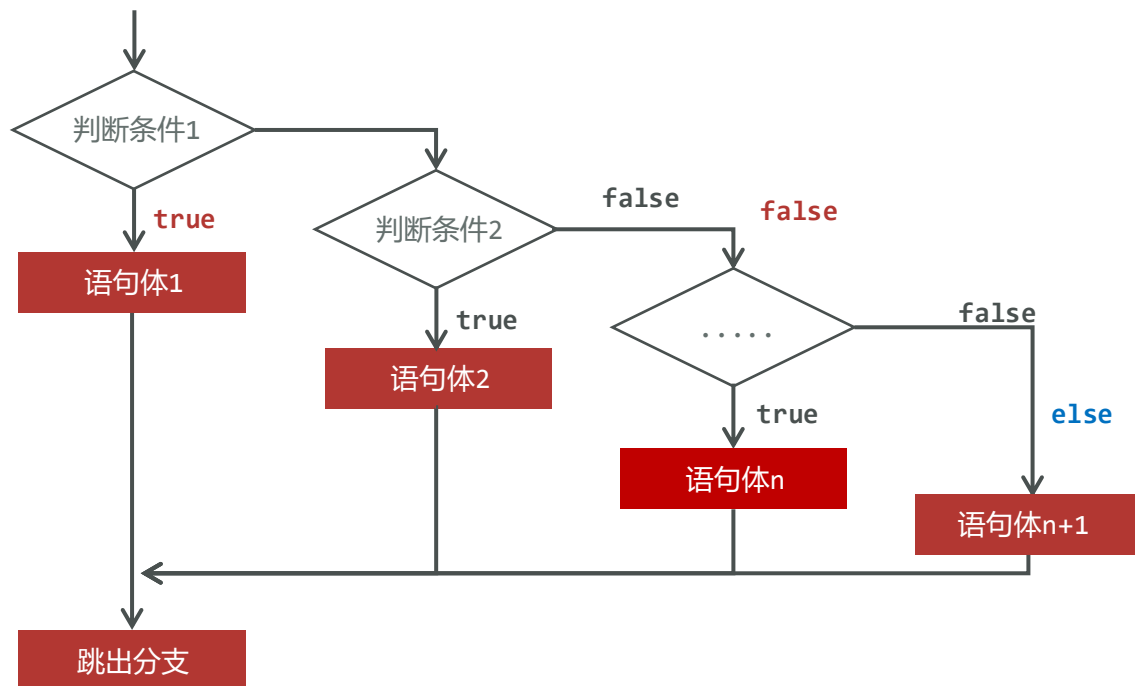
- ① 首先判断条件表达式的结果, 如果为 true 执行语句体 1, 为 false 就执行语句体 2。

if 分支结构

if 第三种形式

格式:

```
if (条件1) {  
    语句体1;  
} else if (条件2) {  
    语句体2;  
} else if (条件3) {  
    语句体4;  
}  
...  
else {  
    语句体n+1;  
}
```



执行流程:

- ① 先判断条件1的值，如果为true则执行语句体1，分支结束；如果为false则判断条件2的值
- ② 如果值为 true 就执行语句体 2，分支结束；如果为 false 则判断条件 3 的值
- ③ ...
- ④ 如果没有任何条件为 true，就执行 else 分支的语句体 n+1。

1、if 分支是什么？举几个应用场景？

- 可以根据条件，选择执行某段程序。

2、if 分支的写法有几种,各有什么特点？

BRIEF 小结

```
if (条件表达式) {  
    语句体;  
}
```

```
if (条件表达式) {  
    语句体1;  
} else {  
    语句体2;  
}
```

```
if (条件表达式1) {  
    语句体1;  
} else if (条件表达式2) {  
    语句体2;  
} else if (条件表达式3) {  
    语句体3;  
}  
...  
else {  
    语句体n+1;  
}
```



练习

自动驾驶汽车过红绿灯

- 假设你正在开发一个自动驾驶汽车项目，该程序如何实现汽车安全通过这个路口？



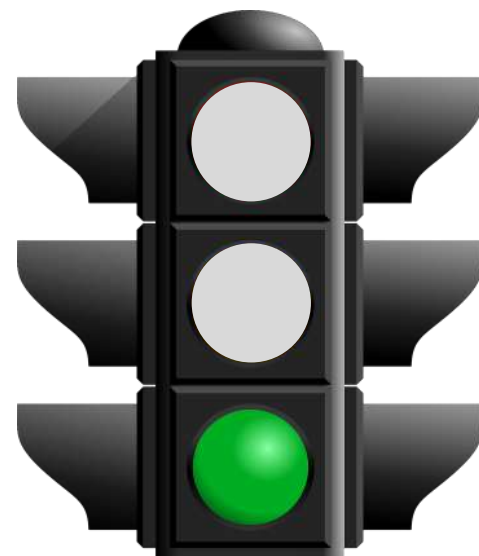
分析:

- 每个灯有亮和不亮两种状态，程序中使用什么类型的变量记录三个灯的状态呢？

boolean

- 如何判断是哪种灯亮？

使用 if 分支结构来判断，从而选择对应的控制程序来控制汽车的行驶状态



- 完成 BMI 后续功能：得到用户的 BMI 指数后，根据 BMI 指数提示出该用户具体的身体健康情况

BMI (Body Mass Index, 身体质量指数) 是用来评估体重是否适宜的一个常用指标。它通过体重 (以千克为单位) 除以身高 (以米为单位) 的平方来计算。

BMI 的正常范围通常定义为 18.5 到 24.9 (kg/m^2)。具体来说：

- BMI 小于 18.5 被认为是体重过低；
- BMI 在 18.5 到 24.9 之间被认为是正常范围；
- BMI 在 25 到 29.9 之间被认为是超重；
- BMI 在 30 及以上被认为是肥胖。

分析:

- 用户的 BMI 健康水平，是按照在每个区间范围来确定的，区间存在多个，建议用什么来解决？

`if... else if... else`

CONTENT

程序流程控制

➤ 分支结构

if

switch

➤ 循环结构

➤ break、continue

➤ 综合小案例





switch 分支结构

- 是通过比较值是否相等，来决定执行哪条分支。



switch 分支结构

- 是通过比较值是否相等，来决定执行哪条分支。

```
switch(表达式){  
    case 值1:  
        执行代码...;  
        break;  
    case 值2:  
        执行代码...;  
        break;  
    ...  
    default:  
        执行代码n;  
}
```

switch 分支的执行流程

- ① 先执行表达式的值，再拿着这个值去与 case 后的值进行匹配。
- ② 与哪个case后的值匹配为true就执行哪个case块的代码，遇到break就跳出switch分支。
- ③ 如果全部 case 后的值与之匹配都是 false，则执行 default 块的代码。

if、switch 的比较：各自适合什么业务场景？

- if 在功能上远远强大于 switch。
- 当前条件是区间的时候，建议使用 if 分支结构来实现
- 当条件是与一个一个的值比较的时候，建议用 switch 更合适（格式良好，性能较好，代码优雅）。



BRIEF 小结

1、switch 分支的格式、执行流程是怎么样的？

2、if、switch 的比较，各自适合什么业务场景？

- if 其实在功能上远远强大于 switch。
- if 适合做条件是区间判断的情况。
- switch 适合做：条件是比较值的情况、代码优雅、性能较好。

```
switch(表达式){  
    case 值1:  
        执行代码...;  
        break;  
    case 值2:  
        执行代码...;  
        break;  
    ...  
    default:  
        执行代码n;  
}
```


switch 的几点注意事项，穿透性的应用

switch 分支结构的几点注意事项

- ① 表达式类型只能是 byte、short、int、char，JDK5 开始支持枚举，JDK7 开始支持 String、不支持 double、float、long。
- ② case 给出的值不允许重复，且只能是字面量，不能是变量。
- ③ 正常使用 switch 的时候，不要忘记写 break，否则会出现穿透现象。

```
switch(表达式){  
    case 值1:  
        执行代码...;  
        break;  
    case 值2:  
        执行代码...;  
        break;  
    ...  
    case 值n-1:  
        执行代码...;  
        break;  
    default:  
        执行代码n;  
}
```



switch 穿透性的应用场景

多学一招：switch 穿透性在些情况下可以简化代码

- 当存在多个 case 分支的代码相同时，可以把相同的代码放到一个 case 块中，其他的 case 块都通过穿透性穿透到该 case 块执行代码即可，这样可以简化代码。

- | | |
|------------------|-----------|
| ● 周一：埋头苦干，解决 bug | 周五：自己整理代码 |
| ● 周二：请求大牛程序员帮忙 | 周六：打游戏 |
| ● 周三：请求大牛程序员帮忙 | 周日：打游戏 |
| ● 周四：请求大牛程序员帮忙 | |

BRIEF 小结

1、使用 switch 时有哪些注意事项？

- 表达式类型**只能是 byte、short、int、char**，JDK5 开始支持枚举，JDK7 开始支持 String、**不支持 double、float、long**。
- case 给出的值不允许重复，且只能是字面量，不能是变量。
- 正常使用 switch 的时候，不要忘记写 break，否则会出现穿透现象。

2、switch 穿透性能解决什么问题？

- 存在多个 case 分支的代码是一样时，可以把代码写到一个 case 块，其他 case 块通过穿透性能，穿透到该 case 块即可，这样可以简化代码。

```
switch(表达式){  
    case 值1:  
        执行代码...;  
        break;  
    case 值2:  
        执行代码...;  
        break;  
    ...  
    case 值n-1:  
        执行代码...;  
        break;  
    default:  
        执行代码n;  
}
```

CONTENT

程序流程控制

➤ 分支结构

➤ 循环结构

➤ break、continue

➤ 综合小案例

◆ for

◆ while

◆ do-while

◆ 死循环

◆ 循环嵌套



循环结构的作用和应用场景

减少代码的重复编写、灵活的控制程序的执行



当前位置: 班级学员管理 > 班级管理

校区	请选择	学科	请选择	班级类型	请选择	授课模式	
开班日期		至		结课日期		至	
	班级名称	开课时间	结课时间	学员数量			
1	广州黑马JavaEE就业170期 (20220325面授)	2022-03-25	2022-08-19(预)	81			
2	北京金燕龙黑马JavaEE就业388期 (20220305面授)	2022-03-05	2022-07-29(预)	82			
3	北京金燕龙黑马JavaEE基础422期 (20220218面授)	2022-02-18	2022-03-03(实)	96			
4	广州黑马JavaEE基础178期 (20211210新型面授)	2021-12-10	2021-12-22(实)	32			
5	武汉黑马JavaEE就业136期 (20211024面授)	2021-10-24	2022-03-29(实)	93			
6	武汉黑马JavaEE基础143期 (20210924面授)	2021-09-24	2021-10-21(实)	49			



for 循环

- 控制一段代码反复执行很多次。

for 循环格式

```
for (初始化语句; 循环条件; 迭代语句) {  
    循环体语句(重复执行的代码);  
}
```

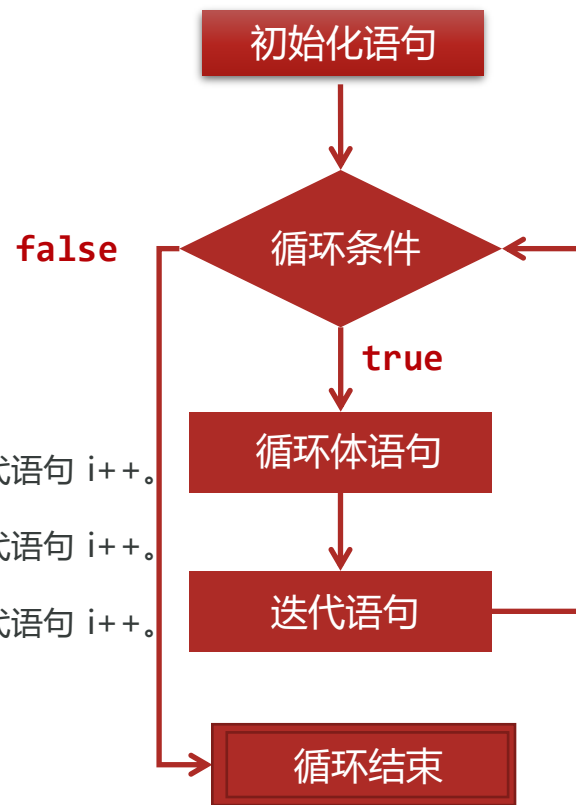
```
// 输出3次HelloWorld  
for (int i = 0; i < 3; i++) {  
    System.out.println("Hello World");  
}
```

for 循环

for 循环案例详细流程说明

```
// 输出3次HelloWorld  
for (int i = 0; i < 3; i++) {  
    System.out.println("Hello World");  
}
```

- ① 循环一开始，执行 `int i = 0` 一次。
- ② 此时 `i=0`，接着计算机执行循环条件语句：`0 < 3` 返回 `true`，计算机就进到循环体中执行，输出：`helloWorld`，然后执行迭代语句 `i++`。
- ③ 此时 `i=1`，接着计算机执行循环条件语句：`1 < 3` 返回 `true`，计算机就进到循环体中执行，输出：`helloWorld`，然后执行迭代语句 `i++`。
- ④ 此时 `i=2`，接着计算机执行循环条件语句：`2 < 3` 返回 `true`，计算机就进到循环体中执行，输出：`helloWorld`，然后执行迭代语句 `i++`。
- ⑤ 此时 `i=3`，然后判断循环条件：`3 < 3` 返回 `false`，循环立即结束！！



BRIEF 小结

for 循环格式和执行流程是什么样的？

```
for (初始化语句 ; 循环条件; 迭代语句) {  
    循环体语句;  
}
```

```
for (int i = 0; i < 3; i++) {  
    System.out.println("Hello World");  
}
```

for 循环的常见应用场景？

- 减少代码的重复编写，灵活的控制程序的执行。



需求：求 1~5 之间的数据和，并把求和结果在控制台输出。

分析：

① 使用 for 循环依次产生：1、2、3、4、5。

```
for (int i = 1; i <= 5; i++) {  
      
}
```

② 在循环外定义一个 int 类型的变量 sum 用于求和，循环每产生一个数，就加到 sum 中去

```
int sum = 0;  
for (int i = 1; i <= 5; i++) {  
    sum += i;  
}
```

int sum = 0;



③ 循环结束后，输出求和变量即是结果。



需求：求 1~10 之间的奇数和，并把求和结果在控制台输出。

分析

- ① 定义 for 循环，使其能够依次产生：1、2、3、4、5...10。
- ② 在循环外定义一个 int 类型的变量 sum 用于求和
- ③ 循环每产生一个数据，都通过 if 判断其是否是奇数，是奇数，则进到 if 中将其加到 sum 中去。

```
for (int i = 1; i <= 10 ; i++) {  
    if(i % 2 == 1){  
        // i = 1 3 5 7 9  
    }  
}
```



需求：求 1~10 之间的奇数和，并把求和结果在控制台输出。

分析：

① 定义 for 循环，使其能够依次产生：1、3、5、7、9

```
for (int i = 1; i <= 10; i+=2) {  
}
```

① 在循环外定义一个整数变量 `sum`，循环每产生一个数据就加到 `sum` 中去。

② 循环结束后，输出 `sum` 变量即可。

需求：在控制台输出所有的“水仙花数”，水仙花数必须满足如下 2 个要求：

1. 水仙花数是一个三位数
2. 水仙花数的个位、十位、百位的数字立方和等于原数

分析：

- ① 定义一个 for 循环从 “100 一直到 999”。
- ② 每次访问到数据后，提取该数据的：个位、十位、百位数字。
- ③ 使用 if 判断：个位、十位、百位的数字立方和是否等于原数，等于则输出该数据。

153 370 371 407



思考

如果还要知道水仙花数的个数怎么办？

- 在循环外定义一个变量 `count` 用于记录水仙花数。
- 每输出水仙花数时，让 `count++`。

BRIEF 小结

1. 如何找出水仙花数?

- 定义一个 for 循环从 “100 一直到 999” 。
- 每次访问到数据后，提取该数据的：个位、十位、百位数字。
- 看各个数的立方和是否等于原数，等于则输出原数据。

2. 如何计算出水仙花的个数?

- 在循环外定义一个变量 count 用于记录水仙花数。
- 每输出水仙花数时，让 count++。

CONTENT

程序流程控制

➤ 分支结构

➤ 循环结构

➤ break、continue

➤ 综合小案例

◆ for

◆ while

◆ do-while

◆ 死循环

◆ 循环嵌套





while 循环

```
初始化语句;  
while (循环条件) {  
    循环体语句(被重复执行的代码);  
    迭代语句;  
}
```

```
int i = 0;  
while (i < 3) {  
    System.out.println("Hello World");  
    i++;  
}
```

执行的流程:

- ① 循环一开始, 执行 `int i = 0` 一次。
- ② 此时 `i=0`, 接着计算机执行循环条件语句: `0 < 3` 返回 `true`, 计算机就进到循环体中执行, 输出: `helloWorld`, 然后执行迭代语句 `i++`。
- ③ 此时 `i=1`, 接着计算机执行循环条件语句: `1 < 3` 返回 `true`, 计算机就进到循环体中执行, 输出: `helloWorld`, 然后执行迭代语句 `i++`。
- ④ 此时 `i=2`, 接着计算机执行循环条件语句: `2 < 3` 返回 `true`, 计算机就进到循环体中执行, 输出: `helloWorld`, 然后执行迭代语句 `i++`。
- ⑤ 此时 `i=3`, 然后判断循环条件: `3 < 3` 返回 `false`, 循环立即结束!!

什么时候用 **for**，什么时候用 **while**？

- 功能上是完全一样的，for 能解决的 while 也能解决，反之亦然。
- 使用规范：知道循环几次使用 for；不知道循环几次建议使用 while。



- 假设你在银行投资了 100000 元，银行给出的复利是 1.7%，问多少年后能实现本金翻倍？



分析

- 初始化投资的本金和目标金额。
- 使用一个变量来跟踪已经经过的年数。
- 在一个 while 循环中，每年根据复利公式更新投资金额，直到它至少翻倍。
- 打印或返回所需的年数

```
double principal = 100000; // 初始本金
double rate = 0.017; // 年利率
double target = 2 * principal; // 目标金额，本金翻倍
int years = 0; // 已经经过的年数

while (principal < target) {
    principal += principal * rate; // 应用复利公式
    years++; // 增加年数
}

System.out.println("需要的年数: " + years);
```

BRIEF 小结

1、while 循环的格式，执行流程是怎么样的？

```
初始化语句;  
while (循环条件) {  
    循环体语句;  
    迭代语句;  
}
```

```
int i = 0;  
while (i < 3) {  
    System.out.println("Hello World");  
    i++;  
}
```

2、while 和 for 有什么区别？什么时候用 for，什么时候用 while？

- 功能上是完全一样的，for 能解决的 while 也能解决，反之亦然。
- 使用规范：知道循环几次使用 for；不知道循环几次建议使用 while。



需求：世界最高山峰珠穆朗玛峰高度是：8848.86 米 = 8848860 毫米，假如我有一张足够大的纸，它的厚度是 0.1 毫米。请问：该纸张折叠多少次，可以折成珠穆朗玛峰的高度？

分析

① 定义变量存储珠穆朗玛峰的高度、纸张的高度。

```
double peakHeight = 8848860; // 山峰高度  
double paperThickness = 0.1; // 纸张厚度
```

② 使用 while 循环来控制纸张折叠，循环条件是（纸张厚度<山峰高度），循环每执行一次，就表示纸张折叠一次，并把纸张厚度变为原来两倍。

③ 循环外定义计数变量 count，循环每折叠一次纸张，让 count 变量 +1；

BRIEF 小结

1、本案例是如何解决的？

- 定义变量存储珠穆朗玛峰的高度、纸张的厚度。
- 使用 while 循环来控制纸张折叠，循环条件是（纸张厚度<山峰高度）
- 循环每执行一次，就表示纸张折叠一次，并把纸张厚度变为原来两倍。
- 循环外定义计数变量 count，循环每折叠一次纸张，让 count 变量 +1；

2、for 和 while 使用总结

- 其实 while 能做的 for 都能实现，for 能做的 while 也都能实现，功能上无区别。
- 使用规范：如果一开始不知道循环次数的情况下，建议使用 while 循环解决更专业。一开始就知道循环几次的情况下，使用 for 循环来解决更专业。

CONTENT

程序流程控制

➤ 分支结构

➤ 循环结构

➤ break、continue

➤ 综合小案例

◆ for

◆ while

◆ do-while

◆ 死循环

◆ 循环嵌套



do-while 循环

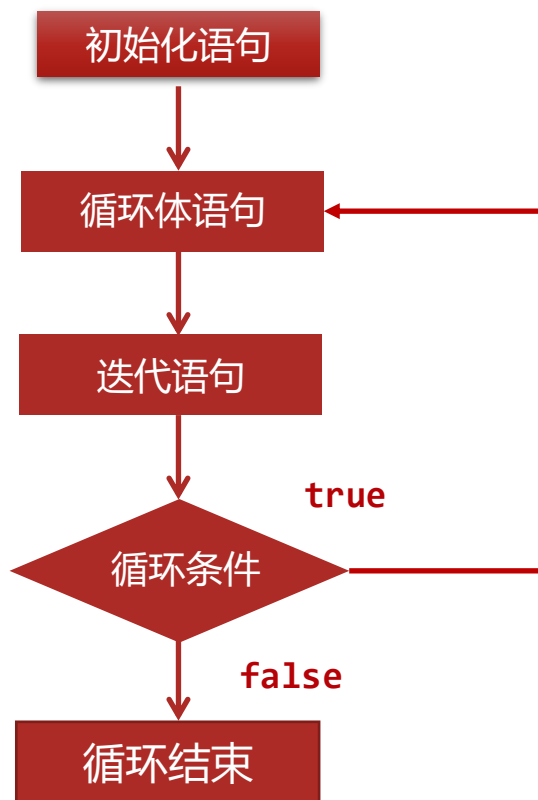
do-while 循环

```
初始化语句;  
do {  
    循环体语句;  
    迭代语句;  
} while (循环条件);
```

```
int i = 0;  
do {  
    System.out.println("Hello World! ");  
    i++;  
} while( i < 3);
```

do-while 循环的特点:

- 先执行后判断。



do-while 循环应用场景示例





三种循环的使用小结

- for 循环和 while 循环（先判断后执行）； do...while（先执行后判断）
- for 循环和 while 循环的执行流程是一模一样的，功能上无区别，for 能做的 while 也能做，反之亦然。
- 使用规范：如果已知循环次数建议使用 for 循环，如果不清楚要循环多少次建议使用 while 循环。
- 其他区别：for 循环中，控制循环的变量只在循环中使用。while 循环中，控制循环的变量在循环后还可以继续使用。

```
for (int i = 0; i < 3; i++ ) {  
    System.out.println( "Hello World");  
}  
System.out.println(i);
```

```
int i = 0;  
while (i < 3) {  
    System.out.println("Hello World");  
    i++;  
}  
System.out.println(i);
```

CONTENT

程序流程控制

➤ 分支结构

➤ 循环结构

➤ break、continue

➤ 综合小案例

◆ for

◆ while

◆ do-while

◆ 死循环

◆ 循环嵌套





死循环

- 可以一直执行下去的一种循环，如果没有干预不会停下来。

死循环的写法

```
for ( ; ; ) {  
    System.out.println("Hello World1");  
}  
// 经典写法  
while (true) {  
    System.out.println("Hello World2");  
}  
  
do {  
    System.out.println("Hello World3");  
} while (true);
```

服务器程序

CONTENT

程序流程控制

➤ 分支结构

➤ 循环结构

➤ break、continue

➤ 综合小案例

◆ for

◆ while

◆ do-while

◆ 死循环

◆ 循环嵌套





循环嵌套

循环嵌套

- 循环中又包含循环。

```
for(int i = 0; i < 3; i++) {  
    for(int j = 0; j < 5; j++) {  
        System.out.println("我爱你");  
    }  
}
```

循环嵌套的特点

- 外部循环每循环一次，内部循环会全部执行完一轮。

```
*****  
*****  
*****  
*****
```

```
for(int i = 1; i <= 4; i++) {  
    System.out.println("*****");  
}
```

100 列呢?



CONTENT

程序流程控制

- 分支结构
- 循环结构
- **break、continue**
- 综合小案例





break、continue

- break: 跳出并结束当前所在循环的执行。
- continue: 用于跳出当前循环的当次执行, 直接进入循环的下一次执行。

注意事项

break: 只能用于结束所在循环, 或者结束所在 switch 分支的执行。

continue: 只能在循环中进行使用。

CONTENT

程序流程控制

➤ 分支结构

➤ 循环结构

➤ break、continue

➤ 综合小案例

◆ 简单计算器

◆ 猜数字游戏

◆ 验证码

◆ 找素数

目标:

- 设计一个可以执行基本数学运算（加、减、乘、除）的计算器程序。

功能描述:

- 用户输入两个数字、一个运算符（+、-、*、/）。
- 根据所选运算符执行相应的数学运算，显示运算结果。

CONTENT

程序流程控制

➤ 分支结构

➤ 循环结构

➤ break、continue

➤ 综合小案例

◆ 简单计算器

◆ 猜数字游戏

◆ 验证码

◆ 找素数

需求:

- 随机生成一个 1~100 之间的数据, 提示用户猜测, 猜大提示过大, 猜小提示过小, 直到猜中结束游戏。

分析:

- ① 先随机生成一个 1~100 之间的数据。
- ② 定义一个死循环让用户可以一直猜测。
- ③ 在死循环里, 每次都提示用户输入一个猜测的数字, 猜大提示过大, 猜小提示过小, 猜中则结束游戏。



Random 生成随机数

- 作用：生成随机数。

得到 0~9 的随机数的实现步骤：

①导包：告诉程序去 JDK 的哪个包中找 Random

```
package com.itheima.random;
```

```
import java.util.Random;
```

```
public class Test {
```

```
    public static void main(String[] args) {
```

```
        Random r = new Random();
```

```
        int number = r.nextInt(10);
```

```
        System.out.println("随机生成了：" +  
number);
```

```
    }
```

```
}
```

②写一行代码拿到随机数对象

③调用随机数的功能获取 0~9 之间的随机数

注意：

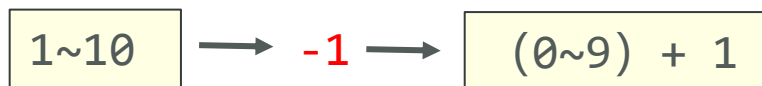
- nextInt(n) 功能只能生成：0 至 n-1 之间的随机数，不包含 n



Random 生成随机数

- 例如：要生成 1 ~ 10 之间的随机数，程序怎么实现？

技巧：减加法



```
Random r = new Random();  
int number = r.nextInt(10) + 1; // 1~10  
              (0~9)
```

BRIEF 小结

Random 生成随机数需要几步？

- 导包: `import java.util.Random;`
- `Random r = new Random();`
- `int number = r.nextInt(10);`

如何生成 65~91 之间的随机数？

- $65 \sim 91 \Rightarrow -65 \Rightarrow (0 \sim 26) + 65$
- `int number = r.nextInt(27) + 65;`

CONTENT

程序流程控制

➤ 分支结构

➤ 循环结构

➤ break、continue

➤ 综合小案例

◆ 简单计算器

◆ 猜数字游戏

◆ 验证码

◆ 找素数

系统登录

验证码, 单击图片刷新



请输入验证码

重置

登录

需求

开发一个程序，可以生成指定位数的验证码，每位可以是数字、大小写字母。

分析：

- ① 使用 for 循环依次生成每位随机字符
- ② 并使用一个 String 类的变量把每个字符连接起来，最后返回该变量即可。



总结

1、随机验证码的核心实现逻辑是如何进行的？

- ① 定义一个 for 循环，循环 5 次。
- ② 随机生成 0|1|2 的数据，依次代表当前要生成的字符是：数字、大写字母、小写字母。
- ③ 把 0、1、2 交给 switch 生成对应类型的随机字符。
- ④ 在循环外定义一个 String 类型的变量用来连接生成的随机字符。
- ⑤ 循环结束后，返回 String 类型的变量即是生成的随机验证码。

CONTENT

程序流程控制

➤ 分支结构

➤ 循环结构

➤ break、continue

➤ 综合小案例

◆ 简单计算器

◆ 猜数字游戏

◆ 验证码

◆ 找素数



Java 笔试题

4. 编程题 (20')

本部分每道题 10 分。

1、判断 101-200 之间有多少个素数，并输出所有素数。

说明：除了 1 和它本身以外，不能被其他正整数整除，就叫**素数**。

比如：3、7 就是素数，而 9、21 等等不是素数。